The PageRank Algorithm

Homework 1 and Homework 3

Computational Linear Algebra for Large Scale Problems
Politecnico di Torino, A.Y. 2025/2026

Ömer Özkan (s360761) Zeynep Sude Ulaş (s364189)

September 2026

Abstract

This report is our work on the paper *The $25,000,000,000 Eigenvector: The Linear Algebra Behind Google* by K. Bryan and T. Leise. Part I is Homework 1: we describe the PageRank algorithm, we explain why it works and when it does not, we present our own implementation (a hand-written sparse matrix and the power method), we test it on the two small webs of the paper and on the Hollins dataset (6012 pages), and we solve Exercises 11 and 17. Part II is Homework 3: we solve ten more exercises of the paper (1, 2, 3, 5, 6, 7, 8, 9, 12 and 13). All numbers in this report are produced by the code in our repository.

Contents

Part I
Homework 1: The PageRank Project

1 Introduction

The task

The homework asks us to read the paper of Bryan and Leise [1], to write our own code for the PageRank algorithm, to test it on the two webs of the paper (Figure 2.1 and Figure 2.2), to test it on the given dataset, and to solve two exercises of our choice. We chose **Exercise 11** (a new page is added to the web and the ranking is computed with the matrix $\mathbf{M}$) and **Exercise 17** (how the damping factor m should be chosen).

The dataset

The dataset is the *Hollins* web graph: 6012 pages of the website of Hollins University and 23 875 links between them. It is stored in the file `hollins.dat`.

Division of the work

Both of us worked on the algorithm, the code and the tests. The exercises were divided as in Table 1. Both of us are ready to explain every part of the report.

Part	**Ömer Özkan**	**Zeynep Sude Ulaş**
Homework 1	Exercises 11, 17	—
Homework 3	Exercises 1, 8, 9, 12	Exercises 2, 3, 5, 6, 7, 13

Table 1: Who solved which exercise

Notation

We use the notation of the paper. A web has n pages, numbered $1,\dots,n$. The vector $\mathbf{x} = (x_1,\dots,x_n)^T$ contains the importance scores, and we always scale it so that $\sum_i x_i = 1$. $V_1(\mathbf{A})$ is the eigenspace of the matrix $\mathbf{A}$ for the eigenvalue 1. When we quote a formula of the paper we use the paper's number, for example "formula (2.1)" or "formula (3.2)".

2 The PageRank Algorithm

2.1 How it works

The idea

A link from page j to page k is a *vote* of page j for page k. A vote from an important page should count more than a vote from an unimportant page. Also, a page should not become powerful only by linking to many pages: each page has one vote in total, and it divides this vote equally among its outgoing links.

Let L_k be the set of pages that link to page k (the *backlinks* of k), and let n_j be the number of outgoing links of page j. The paper defines the score of page k by formula (2.1):

$$x_k = \sum_{j \in L_k} \frac{x_j}{n_j}.$$

The link matrix A

The n equations of formula (2.1) can be written as one matrix equation $\mathbf{Ax} = \mathbf{x}$, where

$$A_{ij} = \begin{cases} 1/n_j & \text{if page } j \text{ links to page } i, \\ 0 & \text{otherwise.} \end{cases}$$

So column j of $\mathbf{A}$ describes the outgoing links of page j, and row i describes the backlinks of page i. Every column sums to one (page j divides its whole vote), so $\mathbf{A}$ is *column-stochastic*. The score vector $\mathbf{x}$ is an eigenvector of $\mathbf{A}$ with eigenvalue 1.

- For the four-page web of Figure 2.1 the paper finds $\mathbf{x} = \frac{1}{31}(12, 4, 9, 6)^T$, so page 1 is the most important.
- Every column-stochastic matrix has 1 as an eigenvalue (Proposition 1 of the paper), so a solution always exists when there are no dangling nodes.

The matrix M

The matrix $\mathbf{A}$ has two problems (see Section 2.3). To fix them the paper mixes $\mathbf{A}$ with the matrix $\mathbf{S}$ whose entries are all $1/n$, using a number $m \in [0, 1]$ called the *damping factor*:

$$\mathbf{M} = (1 - m)\mathbf{A} + m\mathbf{S} \qquad \text{(formula (3.1)).}$$

Google used $m = 0.15$, and we use the same value. Because $\mathbf{Sx} = \mathbf{s}$ when $\sum_i x_i = 1$, where $\mathbf{s}$ is the vector with all entries $1/n$, the equation $\mathbf{Mx} = \mathbf{x}$ can also be written as

$$\mathbf{x} = (1 - m)\mathbf{Ax} + m\mathbf{s} \qquad \text{(formula (3.2)).}$$

Formula (3.2) is the form we use in the code: we never build $\mathbf{M}$, we only multiply by the sparse matrix $\mathbf{A}$ and add the constant vector $m\mathbf{s}$.

The power method

For a web with billions of pages we cannot compute eigenvectors directly. The paper uses the *power method*: start from any positive vector $\mathbf{x}_0$ with $\|\mathbf{x}_0\|_1 = 1$ and repeat

$$\mathbf{x}_{k+1} = \mathbf{M}\mathbf{x}_k = (1 - m)\mathbf{A}\mathbf{x}_k + m\mathbf{s}.$$

The sequence $\mathbf{x}_k$ converges to the score vector $\mathbf{q}$ (Theorem 4.2 of the paper). We stop when two consecutive vectors are close in the 1-norm, $\|\mathbf{x}_{k+1} - \mathbf{x}_k\|_1 < \texttt{tol}$. Each step costs one product $\mathbf{Ax}$, which is cheap because $\mathbf{A}$ is sparse: on Hollins only 0.066% of the entries of $\mathbf{A}$ are non-zero.

2.2 Why it works

The proofs are in Section 3.2 and Section 4 of the paper. We summarise the chain of ideas.

1. **M is column-stochastic** (Exercise 7). So 1 is an eigenvalue of $\mathbf{M}$ and $V_1(\mathbf{M})$ is not empty.
2. **M is positive.** Every entry of $\mathbf{S}$ is $1/n > 0$, so for $m > 0$ every entry $M_{ij} \geq m/n > 0$.
3. **A positive column-stochastic matrix has a one-dimensional** V_1 (Lemma 3.2). Any eigenvector in $V_1(\mathbf{M})$ has all positive or all negative entries (Proposition 2), and two independent vectors could be combined into a vector with mixed signs (Proposition 3), which is impossible. So there is exactly one score vector $\mathbf{q}$ with positive entries and $\sum_i q_i = 1$.
4. **The power method converges to q** (Proposition 5). Write $\mathbf{x}_0 = \mathbf{q} + \mathbf{v}$ with $\sum_j v_j = 0$. Then $\mathbf{M}^k\mathbf{x}_0 = \mathbf{q} + \mathbf{M}^k\mathbf{v}$, and Proposition 4 gives $\|\mathbf{M}^k\mathbf{v}\|_1 \leq c^k\|\mathbf{v}\|_1$ with $c < 1$. So the error goes to zero.
5. **Speed.** The error decreases roughly like $|\lambda_2|^k$, where λ_2 is the second largest eigenvalue of $\mathbf{M}$, and the paper states that $|\lambda_2| \leq 1 - m$. So a larger m gives faster convergence. We check this in Exercise 17.

2.3 When it does not work

Disconnected webs: the ranking of A is not unique

If the web (seen as an undirected graph) has r disconnected pieces, then $\dim V_1(\mathbf{A}) \geq r$ (Section 2.2.1 of the paper). Figure 2.2 has two pieces and $\dim V_1(\mathbf{A}) = 2$: every combination of the two basis vectors is an eigenvector, so $\mathbf{A}$ does not tell us which one is "the" ranking. The matrix $\mathbf{M}$ solves this because it is positive, so $\dim V_1(\mathbf{M}) = 1$ for every web (Exercises 2, 3 and 13).

Dangling nodes: A loses mass

A *dangling node* is a page with no outgoing links. Its column in $\mathbf{A}$ is all zeros, so $\mathbf{A}$ is only *sub*-stochastic and 1 may not be an eigenvalue at all. The paper does not treat this case, but the Hollins dataset has 3189 dangling pages (53% of all pages), so our code must handle it. We use the standard fix: a dangling page gives its vote equally to all n pages. In the code this means adding the number $\frac{1}{n}\sum_{j \text{ dangling}} x_j$ to every entry of $\mathbf{Ax}$. After this fix the matrix is column-stochastic again and the theory above applies.

The value $m = 0$: the power method may not converge

With $m = 0$ we have $\mathbf{M} = \mathbf{A}$, which is not positive. Then the power method can fail. On Figure 2.2 we ran the iteration $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k$ from the starting vector $\mathbf{x}_0 = (0.5, 0.1, 0.2, 0.1, 0.1)^T$: after 1000 steps the residual $\|\mathbf{x}_{k+1} - \mathbf{x}_k\|_1$ was still 1.0, because $\mathbf{A}$ has the eigenvalue -1 and the iterates jump between two vectors. With $m = 0.15$ the same start converges in 143 steps to $(0.2, 0.2, 0.285, 0.285, 0.03)^T$, the vector given in the paper.

The value m close to 1: the links are ignored

When $m \to 1$, $\mathbf{M} \to \mathbf{S}$ and all scores tend to $1/n$. The ranking then says nothing about the web. The choice of m is therefore a compromise between speed and information (Exercise 17).

Link spam

Because a page can raise its score by creating pages that link to it, PageRank can be manipulated. Exercise 1 shows a small example of this.

3 Implementation and Results

The code is written in Python with NumPy only (no SciPy, no sparse library). It is in our repository [2]. The files are listed in Table 2.

File	Content
`toolkit.py`	the matrices, the sparse storage, the power method, the file reader
`webs.py`	every small web used in this report, written as a dictionary
`run_paper.py`	tests against the numbers printed in the paper
`hollins_main.py`	PageRank on the Hollins dataset
`ex01.py, ..., ex17.py`	one script per computational exercise

Table 2: Files of the project

3.1 Data structures

A web is a dictionary

A web is stored as a Python dictionary `{page: [pages it links to]}` with pages numbered $1,\dots,n$ as in the paper. For example Figure 2.1 is

```
FIG_2_1 = {1: [2, 3, 4], 2: [3, 4], 3: [1], 4: [1, 3]}
```

A page with an empty list is a dangling node.

Dense matrix for small webs

The function `build_A` fills an $n \times n$ NumPy array with $A_{ij} = 1/n_j$. We use it only for the small webs, to print $\mathbf{A}$ and $\mathbf{M}$ and to compute all eigenvalues with `numpy.linalg.eig` for cross-checking (for example to find $\dim V_1(\mathbf{A})$). The dense matrix is *not* used by the solver.

Sparse matrix (CSR) for the dataset

For Hollins a dense $\mathbf{A}$ would need $6012^2 \cdot 8$ bytes ≈ 289 MB, but only 23 875 entries are non-zero. We store $\mathbf{A}$ in the *compressed sparse row* format with three arrays, built by hand in `build_csr`:

- `AA`: the non-zero values of $\mathbf{A}$, read row by row;
- `JA`: the column index of each value in `AA`;
- `IA`: for each row i, the position in `AA` where row i starts (length $n+1$).

The non-zeros of row i are `AA[IA[i]:IA[i+1]]`. The three arrays for Hollins take 0.43 MB, about 670 times less than the dense matrix. The function `csr_matvec` computes $\mathbf{y} = \mathbf{A}\mathbf{x}$ row by row:

$$y_i = \sum_{k=\mathtt{IA}[i]}^{\mathtt{IA}[i+1]-1} \mathtt{AA}[k]\, x_{\mathtt{JA}[k]}.$$

We tested that this product equals the dense product `A @ v` on a random vector (difference 0).

3.2 The solver

The only solver of the project is the function `pagerank(links, n, m, tol)`. It starts from $\mathbf{x}_0 = (1/n,\dots,1/n)^T$ and repeats three steps until $\|\mathbf{x}_{k+1} - \mathbf{x}_k\|_1 < \mathtt{tol}$:

Step A. $\mathbf{y} = \mathbf{A}\mathbf{x}_k$ with the CSR product.
Step B. $\mathbf{y} \leftarrow \mathbf{y} + \frac{1}{n}\sum_{j \text{ dangling}} (\mathbf{x}_k)_j$: the mass of the dangling pages is given back, equally to all pages.
Step C. $\mathbf{x}_{k+1} = (1-m)\mathbf{y} + m\mathbf{s}$: this is formula (3.2).

After Step C the sum of $\mathbf{x}_{k+1}$ is already 1 in exact arithmetic; we divide by the sum once more only to remove rounding errors. The function returns the vector $\mathbf{x}$, the number of iterations k and the last residual $\|\mathbf{x}_{k+1}-\mathbf{x}_k\|_1 = \|\mathbf{M}\mathbf{x}_k-\mathbf{x}_k\|_1$. If the residual is not below `tol`, the method did not converge.

With $m = 0$ the same function iterates $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k$, so we also use it to rank a web with formula (2.1) (Exercises 1 and 12).

Reading the dataset

The function `load_dat` reads `hollins.dat`: the first line gives n and the number of links, then n lines give the URL of each page, then one link per line as "source target". Following the paper, a link from a page to itself would be ignored, and a repeated link would be counted once. In the Hollins file there are no self-links and no repeated links.

3.3 Tests on the two webs of the paper

Before using the dataset we checked every number that the paper prints (script `run_paper.py`). Table 3 shows that all of them agree. For each value the allowed difference is one unit of the last printed decimal of the paper.

Web	Quantity checked	Paper	Max. difference
Fig. 2.1	eigenvector of $\mathbf{A} = (12,4,9,6)/31$	p. 3	$1.7\cdot10^{-16}$
Fig. 2.1	$\dim V_1(\mathbf{A}) = 1$	p. 3	exact
Fig. 2.1	matrix $\mathbf{M}$ (four decimals)	Example 1	$3.3\cdot10^{-6}$
Fig. 2.1	scores 0.368, 0.142, 0.288, 0.202	Example 1	$1.9\cdot10^{-4}$
Fig. 2.2	$\dim V_1(\mathbf{A}) = 2$	p. 4	exact
Fig. 2.2	matrix $\mathbf{M}$ of formula (3.3)	p. 6	$5.6\cdot10^{-17}$
Fig. 2.2	scores 0.2, 0.2, 0.285, 0.285, 0.03	Example 2	$5.6\cdot10^{-17}$

Table 3: Checks of our code against the values printed in the paper

The ranking for Figure 2.1 with $\mathbf{M}$ is

$$\mathbf{x} \approx [0.368151,\ 0.141809,\ 0.287962,\ 0.202078]$$

$$(\text{sum} = 1.000)$$

and for Figure 2.2 it is exactly $[0.2,\ 0.2,\ 0.285,\ 0.285,\ 0.03]$, as in Example 2 of the paper. The order of Figure 2.1 is $1 > 3 > 4 > 2$, the same order as with $\mathbf{A}$.

We also tested the dangling-node fix on a small web (Figure 2.1 without the link $3 \to 1$, so page 3 is dangling): the solver and a dense eigenvector computation on the corrected matrix give the same vector (difference $4\cdot10^{-14}$).

3.4 Results on the Hollins dataset

Table 4 summarises the run of `hollins_main.py` with $m = 0.15$ and $\texttt{tol} = 10^{-12}$.

Quantity	Value
pages n	6012
links (non-zeros of $\mathbf{A}$)	23 875
fill of $\mathbf{A}$	0.066%
dangling pages	3189 (53%)
memory, CSR arrays	0.43 MB
memory, dense matrix	289 MB
iterations ($m = 0.15$, $\texttt{tol} = 10^{-12}$)	138
final residual $\|\mathbf{M}\mathbf{x}_k - \mathbf{x}_k\|_1$	$9.7 \cdot 10^{-13}$
time (build CSR + iterations)	1.0 s

Table 4: PageRank on the Hollins dataset

The most important page is the home page of the university, with a score about 120 times larger than the uniform score $1/n = 0.000166$. Table 5 lists the ten best pages. The pages of the admissions area are high because many pages of the site link to them.

Rank	Page	Score	URL
1	2	0.019879	`hollins.edu/`
2	37	0.009288	`hollins.edu/admissions/visit/visit.htm`
3	38	0.008610	`hollins.edu/about/about_tour.htm`
4	61	0.008065	`hollins.edu/htdig/index.html`
5	52	0.008027	`hollins.edu/admissions/info-request/info-request.cfm`
6	43	0.007165	`hollins.edu/admissions/apply/apply.htm`
7	425	0.006583	`hollins.edu/academics/library/resources/web_linx.htm`
8	27	0.005989	`hollins.edu/admissions/admissions.htm`
9	28	0.005572	`hollins.edu/academics/academics.htm`
10	4023	0.004452	`www1.hollins.edu/faculty/saloweyca/clas%20395/...`

Table 5: The ten highest scores on Hollins ($m = 0.15$); the prefix `http://www.` is omitted

Every score is at least $m/n = 2.5 \cdot 10^{-5}$, as formula (3.2) predicts (the smallest score is $5.8 \cdot 10^{-5}$).

Exercise 11: Ranking the Modified Web with the Matrix M

Problem Statement

We take the web of Figure 2.1 and add a page 5 that links to page 3, where page 3 also links to page 5. We are asked to compute the new ranking by finding the eigenvector of $\mathbf{M}$ for $\lambda = 1$ with positive components summing to one, using $m = 0.15$. This is the same web as in Exercise 1, but there the ranking was computed with $\mathbf{A}$; here we use $\mathbf{M}$.

Web and Matrix Construction

The new web has $n = 5$ pages and the links

$$1 \to 2,3,4 \qquad 2 \to 3,4 \qquad 3 \to 1,5 \qquad 4 \to 1,3 \qquad 5 \to 3.$$

- Page 3 now has two outgoing links, so it splits its vote between page 1 and page 5 (entries $1/2$).
- Page 5 has only one link, so it gives its whole vote to page 3 (entry 1).
- The other columns are the same as in Figure 2.1.

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 1/2 & 1/2 & 0 \\ 1/3 & 0 & 0 & 0 & 0 \\ 1/3 & 1/2 & 0 & 1/2 & 1 \\ 1/3 & 1/2 & 0 & 0 & 0 \\ 0 & 0 & 1/2 & 0 & 0 \end{pmatrix},$$

$$\mathbf{M} = 0.85\,\mathbf{A} + 0.03\,\mathbf{1} = \begin{pmatrix} 0.03 & 0.03 & 0.455 & 0.455 & 0.03 \\ 0.3133 & 0.03 & 0.03 & 0.03 & 0.03 \\ 0.3133 & 0.455 & 0.03 & 0.455 & 0.88 \\ 0.3133 & 0.455 & 0.03 & 0.03 & 0.03 \\ 0.03 & 0.03 & 0.455 & 0.03 & 0.03 \end{pmatrix},$$

where $\mathbf{1}$ is the 5×5 matrix of ones and $0.3133 = 0.85/3 + 0.03$. Every column of $\mathbf{M}$ sums to one and every entry is at least $m/n = 0.03 > 0$.

Why the Ranking Is Unique

$\mathbf{M}$ is positive and column-stochastic, so by Lemma 3.2 of the paper $\dim V_1(\mathbf{M}) = 1$. There is exactly one eigenvector for $\lambda = 1$ with positive entries and sum one, and the power method converges to it (Proposition 5).

Computation

We run the power method of Section 3 on formula (3.2) with $m = 0.15$, starting from the uniform vector, with `tol` $= 10^{-12}$. It stops after 57 iterations with residual $6.6 \cdot 10^{-13}$, and gives

$$\mathbf{x} \approx [0.237141,\ 0.097190,\ 0.348894,\ 0.138496,\ 0.178280].$$

$$(\text{sum} = 1.000)$$

The smallest component is $0.097 > 0$, as the theory says. As a cross-check, the eigenvector of the dense matrix $\mathbf{M}$ computed with `numpy.linalg.eig` gives the same vector.

The ranking is $3 > 1 > 5 > 4 > 2$. Table 6 compares it with the ranking of the same web computed with $\mathbf{A}$ in Exercise 1.

Page	Score with A (Ex. 1)	Score with M, $m = 0.15$	Rank
1	0.244898	0.237141	2
2	0.081633	0.097190	5
3	0.367347	0.348894	1
4	0.122449	0.138496	4
5	0.183673	0.178280	3

Table 6: The modified web ranked with **A** and with **M**

The order is the same in both cases. With **M** the scores are a little closer to each other: the large scores go down and the small scores go up, because a part m of every vote is shared equally among all pages.

Conclusion

The new ranking with **M** and $m = 0.15$ is

$$\mathbf{x} \approx [0.2371,\ 0.0972,\ 0.3489,\ 0.1385,\ 0.1783], \qquad 3 > 1 > 5 > 4 > 2.$$

Page 3 is the most important page, as the owners of page 3 wanted (see Exercise 1).

Exercise 17: The Choice of the Damping Factor m

Problem Statement

We are asked how the value of m should be chosen, and how this choice affects the rankings and the computation time. We answer with a short theoretical argument and with an experiment: we rank two webs for $m \in \{0.05,\ 0.15,\ 0.25,\ 0.50,\ 0.85,\ 0.99\}$,

1. the 4-page web of Figure 2.1,
2. the Hollins dataset (6012 pages),

and we record the number of iterations, the time and the ranking (script `ex17.py`). The power method starts from the uniform vector and stops when $\|\mathbf{x}_{k+1} - \mathbf{x}_k\|_1 < \texttt{tol} = 10^{-10}$. We state the tolerance because the number of iterations has no meaning without it.

What the Theory Says

- **Computation time.** The paper states that the error of the power method decreases like $|\lambda_2|^k$, where λ_2 is the second largest eigenvalue of $\mathbf{M}$, and that $|\lambda_2| \le 1 - m$. So after k steps the error is at most about $(1-m)^k$, and to reach a tolerance `tol` we need at most about

$$k \approx \frac{\ln \texttt{tol}}{\ln(1-m)}$$

  iterations. A larger m means a smaller $1-m$ and therefore fewer iterations.
- **Rankings.** Formula (3.2) says $\mathbf{x} = (1-m)\mathbf{A}\mathbf{x} + m\mathbf{s}$: a part $1-m$ of the score comes from the links, a part m is the same for every page. For $m = 0$ the ranking uses only the links, but $\mathbf{M} = \mathbf{A}$ is not positive and the method may not converge (Section 2.3). For $m \to 1$ every score tends to $1/n$ and the ranking becomes meaningless.

So m is a compromise: small m respects the link structure but is slow, large m is fast but forgets the links.

Results: 4-Page Web (Figure 2.1)

Convergence Speed

Even on this tiny web the trend is clear: a larger m needs fewer iterations (Table 7).

m	Iterations	Time (s)	x_1	x_2	x_3	x_4
0.05	35	0.0006	0.380907	0.133120	0.289620	0.196353
0.15	31	0.0004	0.368151	0.141809	0.287962	0.202078
0.25	26	0.0003	0.354915	0.151229	0.285917	0.207940
0.50	18	0.0002	0.320064	0.178344	0.278662	0.222930
0.85	9	0.0002	0.269896	0.225995	0.261165	0.242944
0.99	5	0.0001	0.251256	0.248338	0.250827	0.249579

Table 7: Figure 2.1 for several values of m ($\texttt{tol} = 10^{-10}$); the order is always $1 > 3 > 4 > 2$

Rankings

The order $1 > 3 > 4 > 2$ never changes. What changes is the distance between the scores: for $m = 0.05$ page 1 has 0.381 and page 2 has 0.133, for $m = 0.99$ all four scores are within 0.003 of the uniform value 0.25.

Results: Hollins Dataset (6012 Pages)

Convergence Speed

On the large web the saving is important: going from $m = 0.15$ to $m = 0.85$ divides the number of iterations by ten (Table 8). The last column is the estimate $\lceil \ln \texttt{tol} / \ln(1-m) \rceil$ from the theory; the measured numbers are always below it, so the bound $|\lambda_2| \le 1 - m$ is confirmed and is quite close on this dataset.

m	Iterations	Time (s)	Top 5 pages	$\lceil \ln \texttt{tol} / \ln(1-m) \rceil$
0.05	344	2.50	2, 37, 38, 61, 52	449
0.15	111	0.79	2, 37, 38, 61, 52	142
0.25	65	0.48	2, 37, 38, 52, 425	81
0.50	28	0.22	2, 425, 37, 38, 52	34
0.85	11	0.09	2, 425, 37, 38, 52	13
0.99	5	0.05	2, 425, 37, 38, 52	5

Table 8: Hollins for several values of m ($\texttt{tol} = 10^{-10}$, uniform start)

Rankings

The home page (page 2) is first for every m, but the rest of the top five moves. Page 425 (a library page with a list of web links) is 7th for $m = 0.15$ and 2nd for $m \ge 0.50$. We can explain this with formula (3.2). When m is large all scores are close to $1/n$, so the score of page i is approximately

$$x_i \approx \frac{1-m}{n} \sum_{j \in L_i} \frac{1}{n_j} + \frac{m}{n},$$

and the ranking is decided by the sum $\sum_{j \in L_i} 1/n_j$: the number of backlinks, each weighted by one over the number of links of the page that gives it. Page 425 has only 87 backlinks, but 57 of them come from pages with one or two links, so its sum is 57.9; page 37 has 454 backlinks but its sum is only 28.4. For large m the order of this sum is $2 > 425 > 37 > 38 > 52$, exactly the ranking we measure for $m = 0.99$. For small m the *quality* of the backlinks matters more, and page 425 goes down.

Conclusion

A larger m makes the power method faster (about $\ln \texttt{tol} / \ln(1-m)$ iterations at most) but moves every score towards $1/n$, so the ranking loses the information of the links. On a small web the order does not change; on Hollins the top pages change already for $m = 0.25$. The value $m = 0.15$ used by Google is a reasonable compromise: the ranking follows the link structure, and the method converges in about a hundred iterations on a web of six thousand pages.

Part II
Homework 3: Ten Exercises from the Paper

About Part II

In this part we solve ten exercises of the paper: 1, 2, 3, 5, 6, 7, 8, 9, 12 and 13. Each exercise is written as a short independent note with a problem statement, the solution and a conclusion. The exercises with a computation (1, 2, 3, 12, 13) have a script in the repository with the same number; the others (5, 6, 7, 8, 9) are proofs. Every numerical result was computed with the same solver as in Part I.

Note on the numbering of the figures: Exercise 1 says “the web of Figure 1” and Exercise 3 says “the web of Figure 2”; in the paper these figures are labelled Figure 2.1 and Figure 2.2. We use the labels 2.1 and 2.2.

Exercise 1: Can the Owners of Page 3 Boost Their Score?

Problem Statement

In the web of Figure 2.1 the score of page 3, computed with formula (2.1), is lower than the score of page 1. The owners of page 3 create a new page 5 that links to page 3, and page 3 also links to page 5. We are asked whether this boosts the score of page 3 above the score of page 1. Here the scores are computed with $\mathbf{A}$ only (formula (2.1)), so no damping.

Before: the Web of Figure 2.1

The paper gives the eigenvector $\mathbf{x} = \frac{1}{31}(12, 4, 9, 6)^T$:

$$x_1 \approx 0.387, \qquad x_3 \approx 0.290.$$

Page 1 is first and page 3 is third.

After: Matrix Construction

The new web has $n = 5$ pages.

- Page 5 links only to page 3, so it gives its whole vote to page 3 (entry 1 in column 5).
- Page 3 now links to page 1 and page 5, so it splits its vote (entries $1/2$ in column 3).
- Nothing else changes.

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 1/2 & 1/2 & 0 \\ 1/3 & 0 & 0 & 0 & 0 \\ 1/3 & 1/2 & 0 & 1/2 & 1 \\ 1/3 & 1/2 & 0 & 0 & 0 \\ 0 & 0 & 1/2 & 0 & 0 \end{pmatrix}.$$

Eigenvector Analysis

We solve $\mathbf{Ax} = \mathbf{x}$. The vector $\mathbf{v} = (12, 4, 18, 6, 9)^T$ satisfies it, as we check row by row:

$$\tfrac{18}{2} + \tfrac{6}{2} = 12, \quad \tfrac{12}{3} = 4, \quad \tfrac{12}{3} + \tfrac{4}{2} + \tfrac{6}{2} + 9 = 18, \quad \tfrac{12}{3} + \tfrac{4}{2} = 6, \quad \tfrac{18}{2} = 9.$$

The web is strongly connected, so $\dim V_1(\mathbf{A}) = 1$ and this is the only ranking (Exercise 10 of the paper; `numpy` confirms that 1 is a simple eigenvalue). Scaling to sum one gives

$$\mathbf{x} = \tfrac{1}{49}(12, 4, 18, 6, 9)^T \approx [0.244898,\ 0.081633,\ 0.367347,\ 0.122449,\ 0.183673].$$

$$(\text{sum} = 1.000)$$

Our power method with $m = 0$ (script `ex01.py`) gives the same numbers after 84 iterations.

Table 9 compares the two webs. We also show the unscaled values, because they explain what happens.

Page	Before (unscaled)	Before	After (unscaled)	After
1	12	0.387097	12	0.244898
2	4	0.129032	4	0.081633
3	9	0.290323	**18**	**0.367347**
4	6	0.193548	6	0.122449
5	—	—	9	0.183673
sum	31	1	49	1

Table 9: Scores of Figure 2.1 before and after the new page 5

- **Feedback loop.** Page 3 gives half of its score to page 5, and page 5 gives all of it back to page 3. So page 3 keeps what it had (the votes of pages 1, 2 and 4 are unchanged: $4+2+3=9$) and adds the vote of page 5, which is $18/2=9$. Its unscaled value doubles from 9 to 18.
- **Page 1 does not lose votes.** Its unscaled value stays 12. Before, it received the whole vote of page 3, that is 9. Now it receives only half of the vote of page 3, but page 3 is twice as big, so this is again $18/2=9$; plus 3 from page 4 as before. Its scaled score goes down only because the total grows from 31 to 49.

Conclusion

Yes. After the trick, page 3 has score $18/49 \approx 0.367$ and page 1 has $12/49 \approx 0.245$, so page 3 is now the most important page. This shows that formula (2.1) can be manipulated by adding pages.

Exercise 2: Dimension of $V_1(\mathbf{A})$ for a Web with Three Subwebs

Problem Statement

We are asked to construct a web consisting of three or more disconnected subwebs and to verify that $\dim V_1(\mathbf{A})$ equals (or exceeds) the number of subwebs.

Web and Matrix Construction

We take six pages divided into three subwebs,

$$W_1 = \{1,2\}, \qquad W_2 = \{3,4\}, \qquad W_3 = \{5,6\},$$

where inside each subweb the two pages link to each other ($1 \leftrightarrow 2$, $3 \leftrightarrow 4$, $5 \leftrightarrow 6$) and there is no link between different subwebs. Each page has exactly one outgoing link, so every non-zero entry of $\mathbf{A}$ is 1:

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} \mathbf{A}_1 & 0 & 0 \\ 0 & \mathbf{A}_2 & 0 \\ 0 & 0 & \mathbf{A}_3 \end{pmatrix}, \qquad \mathbf{A}_1 = \mathbf{A}_2 = \mathbf{A}_3 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

The matrix is block diagonal, one block for each subweb.

Eigenvector Analysis

Each block is column-stochastic and satisfies $\mathbf{A}_i(1,1)^T = (1,1)^T$, so $(1,1)^T$ is an eigenvector of each block for the eigenvalue 1. We extend these vectors to the full matrix by putting zeros in the other blocks:

$$\mathbf{w}_1 = (1,1,0,0,0,0)^T, \qquad \mathbf{w}_2 = (0,0,1,1,0,0)^T, \qquad \mathbf{w}_3 = (0,0,0,0,1,1)^T.$$

Because $\mathbf{A}$ is block diagonal, $\mathbf{A}\mathbf{w}_i = \mathbf{w}_i$ for $i = 1,2,3$, so all three vectors are in $V_1(\mathbf{A})$. They are linearly independent, because their non-zero entries are in different positions. Therefore

$$\dim V_1(\mathbf{A}) \geq 3.$$

In fact the dimension is exactly 3: the eigenvalues of $\mathbf{A}$ computed by `numpy` (script `ex02.py`) are $1,-1,1,-1,1,-1$, so the eigenvalue 1 appears three times.

Conclusion

For a web with three disconnected subwebs we found

$$\dim V_1(\mathbf{A}) = 3,$$

equal to the number of subwebs. This confirms the statement of Section 2.2.1 of the paper: with r subwebs, $\dim V_1(\mathbf{A}) \geq r$, and so the ranking given by $\mathbf{A}$ is not unique.

Exercise 3: A Connected Web Whose Ranking Is Still Not Unique

Problem Statement

We add a link from page 5 to page 1 in the web of Figure 2.2. The new web, seen as an undirected graph, is connected. We are asked for the dimension of $V_1(\mathbf{A})$.

Web and Matrix Construction

In Figure 2.2 the links are $1 \leftrightarrow 2$, $3 \leftrightarrow 4$ and $5 \to 3$, $5 \to 4$. After adding $5 \to 1$, page 5 has three outgoing links, so it gives $1/3$ of its vote to each of the pages 1, 3 and 4:

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 0 & 1/3 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1/3 \\ 0 & 0 & 1 & 0 & 1/3 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Only column 5 changed with respect to the matrix of Figure 2.2. Row 5 is still zero: nobody links to page 5.

Eigenvector Analysis

We solve $\mathbf{Ax} = \mathbf{x}$ by writing the five equations:

$$x_1 = x_2 + \tfrac{1}{3}x_5, \qquad x_2 = x_1, \qquad x_3 = x_4 + \tfrac{1}{3}x_5, \qquad x_4 = x_3, \qquad x_5 = 0.$$

The last equation gives $x_5 = 0$, so the new link carries no weight at all. The system reduces to $x_1 = x_2$ and $x_3 = x_4$, and the general solution is

$$\mathbf{x} = a\,(1,1,0,0,0)^T + b\,(0,0,1,1,0)^T, \qquad a,b \in \mathbb{R}.$$

The two vectors are linearly independent, so $\dim V_1(\mathbf{A}) = 2$. The eigenvalues computed by `numpy` (script `ex03.py`) are $1,-1,1,-1,0$: the eigenvalue 1 appears twice, which confirms the result.

Conclusion

$$\dim V_1(\mathbf{A}) = 2.$$

Although the web is now connected as an undirected graph, the ranking given by $\mathbf{A}$ is still not unique. The reason is the direction of the links: page 5 sends votes but receives none, so its score is zero and it cannot connect the two subwebs $\{1,2\}$ and $\{3,4\}$ in the equations. Being connected as an undirected graph is therefore not enough for $\dim V_1(\mathbf{A}) = 1$; the paper's condition of a *strongly* connected web (Exercise 10) is what matters.

Exercise 5: Importance Score of a Page with No Backlinks

Problem Statement

We are asked to prove that, in any web, the importance score of a page with no backlinks is zero. Here the score is defined by formula (2.1), that is, by the matrix $\mathbf{A}$.

Proof

By formula (2.1) the score of page k is

$$x_k = \sum_{j \in L_k} \frac{x_j}{n_j},$$

where L_k is the set of pages that link to page k. If page k has no backlinks, then $L_k = \varnothing$. A sum over the empty set is zero, so

$$x_k = 0.$$

In matrix language: page k has no backlinks means that row k of $\mathbf{A}$ is zero, so $x_k = (\mathbf{A}\mathbf{x})_k = 0$ for every $\mathbf{x} \in V_1(\mathbf{A})$.

Conclusion

With formula (2.1), a page that nobody links to has importance score $x_k = 0$, whatever the rest of the web looks like. Exercise 9 shows that with the matrix $\mathbf{M}$ this score becomes $m/n > 0$.

Exercise 6: The Scores Do Not Depend on the Numbering of the Pages

Problem Statement

We are asked to prove that the way the pages are numbered has no effect on the importance scores. Let $\mathbf{A}$ be the link matrix of a web with n pages, and let $\tilde{\mathbf{A}}$ be the link matrix of the same web after the indices of pages i and j are exchanged. Let $\mathbf{P}$ be the permutation matrix obtained by exchanging rows i and j of the identity matrix.

Proof

Step 1: $\tilde{\mathbf{A}} = \mathbf{PAP}$

Multiplying on the left, $\mathbf{PA}$, exchanges rows i and j of $\mathbf{A}$. Multiplying on the right, $\mathbf{AP}$, exchanges columns i and j. When we rename page i as page j and vice versa, the row of page i becomes the row of page j and the column of page i becomes the column of page j. So both the rows and the columns are exchanged, which is exactly $\mathbf{PAP}$. Also $\mathbf{P}^2 = \mathbf{I}$, because exchanging twice gives the identity, and therefore $\mathbf{P}^{-1} = \mathbf{P}$.

Step 2: $\mathbf{y} = \mathbf{Px}$ is an eigenvector of $\tilde{\mathbf{A}}$

Suppose $\mathbf{Ax} = \lambda\mathbf{x}$ and define $\mathbf{y} = \mathbf{Px}$. Then

$$\tilde{\mathbf{A}}\mathbf{y} = (\mathbf{PAP})(\mathbf{Px}) = \mathbf{PA}(\mathbf{P}^2)\mathbf{x} = \mathbf{PAx} = \mathbf{P}(\lambda\mathbf{x}) = \lambda\,\mathbf{Px} = \lambda\mathbf{y}.$$

So $\mathbf{y}$ is an eigenvector of $\tilde{\mathbf{A}}$ with the same eigenvalue λ.

Step 3: the scores are unchanged

Take $\lambda = 1$ and let $\mathbf{x}$ be the score vector, $\mathbf{Ax} = \mathbf{x}$. Then $\mathbf{y} = \mathbf{Px}$ is the score vector of the renamed web. But $\mathbf{Px}$ is just $\mathbf{x}$ with the entries i and j exchanged: the numbers are the same, only their positions move. So page i (now called page j) has the same score as before. Any permutation of the n indices is a product of such exchanges, so the same holds for any renumbering: if $\mathbf{Q}$ is a permutation matrix, $\hat{\mathbf{A}} = \mathbf{QAQ}^{-1}$ and $\hat{\mathbf{A}}(\mathbf{Qx}) = \mathbf{QAx} = \mathbf{Qx}$.

Conclusion

Renumbering the pages permutes the entries of the score vector in the same way, and the scores themselves do not change. The importance scores are therefore independent of the numbering of the pages.

Exercise 7: The Matrix M Is Column-Stochastic

Problem Statement

We are asked to prove that if $\mathbf{A}$ is an $n \times n$ column-stochastic matrix and $0 \leq m \leq 1$, then

$$\mathbf{M} = (1-m)\mathbf{A} + m\mathbf{S}$$

is also column-stochastic. Here $\mathbf{S}$ is the matrix with all entries equal to $1/n$.

Proof

A matrix is column-stochastic when all its entries are non-negative and every column sums to one. Both $\mathbf{A}$ and $\mathbf{S}$ have these properties:

$$a_{ij} \geq 0, \quad s_{ij} = \tfrac{1}{n} \geq 0, \qquad \sum_{i=1}^{n} a_{ij} = 1, \quad \sum_{i=1}^{n} s_{ij} = n \cdot \tfrac{1}{n} = 1 \quad \text{for every } j.$$

Non-negative entries. The entries of $\mathbf{M}$ are $M_{ij} = (1-m)a_{ij} + ms_{ij}$. Since $0 \leq m \leq 1$, both $1-m \geq 0$ and $m \geq 0$, so M_{ij} is a sum of two non-negative numbers and $M_{ij} \geq 0$.

Column sums. For every column j,

$$\sum_{i=1}^{n} M_{ij} = (1-m)\sum_{i=1}^{n} a_{ij} + m\sum_{i=1}^{n} s_{ij} = (1-m)\cdot 1 + m \cdot 1 = 1.$$

Conclusion

All entries of $\mathbf{M}$ are non-negative and every column of $\mathbf{M}$ sums to one, so $\mathbf{M}$ is column-stochastic. By Proposition 1 of the paper, 1 is then an eigenvalue of $\mathbf{M}$. If moreover $m > 0$, every entry satisfies $M_{ij} \geq m/n > 0$, so $\mathbf{M}$ is positive and Lemma 3.2 gives $\dim V_1(\mathbf{M}) = 1$.

Exercise 8: The Product of Two Column-Stochastic Matrices

Problem Statement

We are asked to show that the product of two column-stochastic matrices is also column-stochastic.

Definitions

Let $\mathbf{B}$ and $\mathbf{C}$ be $n \times n$ column-stochastic matrices:

$$b_{ik} \geq 0, \quad c_{kj} \geq 0, \qquad \sum_{i=1}^{n} b_{ik} = 1, \quad \sum_{k=1}^{n} c_{kj} = 1 \qquad \text{for all } i, j, k.$$

The entries of the product are $(\mathbf{BC})_{ij} = \sum_{k=1}^{n} b_{ik} c_{kj}$.

Proof

Non-negative entries. Each term $b_{ik}c_{kj}$ is a product of two non-negative numbers, so $(\mathbf{BC})_{ij} \geq 0$.

Column sums. Fix a column j and sum over i. We exchange the order of the two sums:

$$\sum_{i=1}^{n} (\mathbf{BC})_{ij} = \sum_{i=1}^{n} \sum_{k=1}^{n} b_{ik} c_{kj} = \sum_{k=1}^{n} c_{kj} \left(\sum_{i=1}^{n} b_{ik} \right) = \sum_{k=1}^{n} c_{kj} \cdot 1 = 1.$$

In the third step we used that every column k of $\mathbf{B}$ sums to one, and in the last step that column j of $\mathbf{C}$ sums to one.

Another way to see it: with $\mathbf{e} = (1, \ldots, 1)^T$, a matrix $\mathbf{B}$ is column-stochastic exactly when $\mathbf{B}^T\mathbf{e} = \mathbf{e}$ and $\mathbf{B} \geq 0$. Then $(\mathbf{BC})^T\mathbf{e} = \mathbf{C}^T\mathbf{B}^T\mathbf{e} = \mathbf{C}^T\mathbf{e} = \mathbf{e}$.

Why This Matters

By induction, every power $\mathbf{A}^k$ of a column-stochastic matrix is column-stochastic, and so is $\mathbf{M}^k$. This is used in the last part of Exercise 10 of the paper, where the matrix $\mathbf{B} = \frac{1}{n}(\mathbf{I} + \mathbf{A} + \cdots + \mathbf{A}^{n-1})$ must be column-stochastic. It also means that in the power method the vector $\mathbf{x}_k = \mathbf{M}^k\mathbf{x}_0$ keeps the sum $\sum_i (\mathbf{x}_k)_i = 1$ at every step, so no rescaling is needed in exact arithmetic.

Conclusion

If $\mathbf{B}$ and $\mathbf{C}$ are column-stochastic, then $\mathbf{BC}$ has non-negative entries and columns that sum to one: $\mathbf{BC}$ is column-stochastic.

Exercise 9: A Page with No Backlinks Gets the Score m/n

Problem Statement

We are asked to show that, with formula (3.2), a page with no backlinks gets the importance score m/n. This is the version with damping of Exercise 5, where the same page got the score 0.

Definitions

Formula (3.2) is

$$\mathbf{x} = (1-m)\mathbf{A}\mathbf{x} + m\mathbf{s},$$

where $\mathbf{s}$ is the vector with all entries $1/n$ and $\mathbf{x}$ is scaled so that $\sum_i x_i = 1$. Page i has no backlinks when no page links to it, that is, when row i of $\mathbf{A}$ is zero.

Proof

Take row i of formula (3.2):

$$x_i = (1-m)\sum_{j=1}^{n} A_{ij}x_j + \frac{m}{n}.$$

If page i has no backlinks, then $A_{ij} = 0$ for every j, so the sum is zero and

$$x_i = \frac{m}{n}.$$

The result does not depend on the rest of the web, only on m and n. Also, since $\mathbf{A}\mathbf{x} \geq 0$, every page gets at least m/n: this is the smallest possible score with the matrix $\mathbf{M}$.

Check with Our Computations

- In Exercise 12, page 6 has no backlinks, $n = 6$, $m = 0.15$, and we compute $x_6 = 0.025000 = 0.15/6$.
- In Exercise 13, page 5 has no backlinks, $n = 5$, and we compute $x_5 = 0.030000 = 0.15/5$.
- In Example 2 of the paper (Figure 2.2), page 5 has no backlinks and the paper gives $x_5 = 0.03 = 0.15/5$.
- On Hollins every score is at least $m/n = 2.5 \cdot 10^{-5}$ (Section 3.4).

A Remark on the Paper

In Section 3.1 the paper writes "a web page with no backlinks (a dangling node)". These are two different things: a page with no backlinks has a zero *row* in $\mathbf{A}$ (nobody links to it), while a dangling node has a zero *column* (it links to nobody). Exercise 9 is about the zero row. For example page 5 of Figure 2.2 has no backlinks but is not dangling, since it links to pages 3 and 4.

Conclusion

With formula (3.2) a page with no backlinks has importance score exactly

$$x_i = \frac{m}{n},$$

instead of 0 as with formula (2.1). The damping term gives every page a small base score.

Exercise 12: A Sixth Page That Links to Everyone

Problem Statement

We add to the web of Exercise 11 a sixth page that links to every other page, but to which no page links. We are asked to rank the pages with **A** and then with **M** ($m = 0.15$), and to compare the results.

Web and Matrix Construction

The new web has $n = 6$ pages.

- Page 6 has five outgoing links, so column 6 has the entry $1/5$ in rows 1 to 5.
- Nobody links to page 6, so row 6 is zero.
- Page 6 is *not* a dangling node: it has outgoing links, so its column sums to one. What it lacks is backlinks.

$$\mathbf{A} = \begin{pmatrix} 0 & 0 & 1/2 & 1/2 & 0 & 1/5 \\ 1/3 & 0 & 0 & 0 & 0 & 1/5 \\ 1/3 & 1/2 & 0 & 1/2 & 1 & 1/5 \\ 1/3 & 1/2 & 0 & 0 & 0 & 1/5 \\ 0 & 0 & 1/2 & 0 & 0 & 1/5 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Ranking with A and with M

Both rankings are computed with the same solver (script `ex12.py`): $m = 0$ gives formula (2.1), $m = 0.15$ gives formula (3.2). Table 10 shows that the order of the pages is the same in both cases, and that only page 6 changes its nature: from 0 to m/n.

Page	Score with A	Score with M, $m = 0.15$	Rank
1	0.244898	0.231212	2
2	0.081633	0.094760	5
3	0.367347	0.340172	1
4	0.122449	0.135033	4
5	0.183673	0.173823	3
6	**0.000000**	**0.025000**	6

Table 10: The six-page web ranked with **A** and with **M**

- **With A.** Page 6 has no backlinks, so $x_6 = 0$ (Exercise 5). Its column then multiplies $x_6 = 0$ and has no effect: the other five scores are exactly those of Exercise 1, $\frac{1}{49}(12, 4, 18, 6, 9)$.
- **With M.** Page 6 gets exactly $x_6 = m/n = 0.15/6 = 0.025$ (Exercise 9). The other five scores are 0.975 times the scores of Exercise 11, for example $0.975 \times 0.237141 = 0.231212$. The reason is simple: page 6 gives $\frac{1}{5}x_6$ to every page, the same amount to all, just like the term $m\mathbf{s}$. So pages 1 to 5 satisfy the same equations as in Exercise 11 with a different constant, and the solution of such a system is proportional to the constant. The proportions do not change, and the sum of the five scores must be $1 - 0.025 = 0.975$.

Conclusion

The order is $3 > 1 > 5 > 4 > 2 > 6$ with both matrices. With **A** the new page scores 0; with **M** it scores $m/n = 0.025$, and the other pages keep their relative importance. Linking to everyone does not help a page that nobody links to: the score comes from the backlinks, not from the outgoing links.

Exercise 13: Ranking a Web with Two Subwebs Using M

Problem Statement

We are asked to construct a web consisting of two or more subwebs and to determine the ranking given by formula (3.1), that is, by the matrix $\mathbf{M} = (1-m)\mathbf{A} + m\mathbf{S}$. We use $m = 0.15$.

Web and Matrix Construction

We take five pages in two disconnected subwebs,

$$W_1 = \{1,2\}, \qquad W_2 = \{3,4,5\},$$

with the links $1 \to 2$, $2 \to 1$, $3 \to 4$, $4 \to 3$ and $5 \to 3$. There is no link between W_1 and W_2, and every page has exactly one outgoing link, so

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}, \qquad \mathbf{M} = 0.85\,\mathbf{A} + 0.03\,\mathbf{1} = \begin{pmatrix} 0.03 & 0.88 & 0.03 & 0.03 & 0.03 \\ 0.88 & 0.03 & 0.03 & 0.03 & 0.03 \\ 0.03 & 0.03 & 0.03 & 0.88 & 0.88 \\ 0.03 & 0.03 & 0.88 & 0.03 & 0.03 \\ 0.03 & 0.03 & 0.03 & 0.03 & 0.03 \end{pmatrix},$$

where $\mathbf{1}$ is the 5×5 matrix of ones and $0.03 = m/n$.

Why A Is Not Enough

The web has two subwebs, so $\dim V_1(\mathbf{A}) = 2$ (the eigenvalues of $\mathbf{A}$ are $1, -1, 1, -1, 0$, script `ex13.py`). Any combination $a\,(1,1,0,0,0)^T + b\,(0,0,1,1,0)^T$ is an eigenvector, so $\mathbf{A}$ gives no unique ranking and cannot compare a page of W_1 with a page of W_2.

Ranking with M

$\mathbf{M}$ is positive, so $\dim V_1(\mathbf{M}) = 1$ (Lemma 3.2) and the ranking is unique. Solving $\mathbf{Mx} = \mathbf{x}$ with $\sum_i x_i = 1$ gives

$$\mathbf{x} = \left[\tfrac{1}{5},\ \tfrac{1}{5},\ \tfrac{54}{185},\ \tfrac{1029}{3700},\ \tfrac{3}{100}\right] \approx [0.200000,\ 0.200000,\ 0.291892,\ 0.278108,\ 0.030000].$$

$$(\text{sum} = 1.000)$$

Our power method gives the same vector after 165 iterations (`tol` $= 10^{-12}$).

- Pages 1 and 2 tie exactly: the subweb $1 \leftrightarrow 2$ is symmetric, and formula (3.2) gives $x_1 = 0.85\,x_2 + 0.03$ and $x_2 = 0.85\,x_1 + 0.03$, so $x_1 = x_2 = 0.2$.
- Page 5 has no backlinks, so its score is exactly $m/n = 0.03$ (Exercise 9).
- Page 3 is above page 4 because it receives the votes of both page 4 and page 5, while page 4 receives only the vote of page 3.

Conclusion

The ranking given by formula (3.1) is

$$3 > 4 > 1 = 2 > 5.$$

Although the web consists of two disconnected subwebs, the matrix $\mathbf{M}$ produces a unique ranking and makes the pages of different subwebs comparable.

References

[1] K. Bryan and T. Leise, *The $25,000,000,000 Eigenvector: The Linear Algebra Behind Google*, SIAM Review, 48 (2006), pp. 569–581.

[2] Ö. Özkan and Z. S. Ulaş, *pagerank_project* (source code), `https://github.com/omerzkan/pagerank_project`.